

AI-Powered Intrusion Detection System

Classifying RPL Routing Attacks in IoT Networks: Machine Learning
Project Report

Models: Logistic Regression, Random Forest, Gradient Boosting

Dataset: IoT-RPL 2021 (Mendeley Data, DOI 10.17632/4rcbbry2sc.1)

Table of Contents

1. Introduction

2. Dataset

2.1 Source and Citation

2.2 Structure and Class Distribution

2.3 Scope of This Implementation

3. Implementation: Preprocessing Pipeline

3.1 Define

3.2 Load the Data

3.3 Explore the Data

3.4 Clean the Data

3.5 Feature Selection and Engineering

3.6 Detecting and Removing Data Leakage

4. Model Implementation

4.1 Baseline Model: Logistic Regression

4.2 Random Forest

4.3 Gradient Boosting

5. Results and Model Comparison

6. Error Analysis and Diagnosis

7. Key Takeaways

1. Introduction

IoT devices rely on RPL (Routing Protocol for Low-power and Lossy Networks) to build their routing tree. RPL is a known target for several attacks. This project builds and compares three models, Logistic Regression, a Random Forest, and Gradient Boosting, to classify a single simulated RPL control message as Normal traffic or one of four attacks. This is a 5-class supervised classification problem.

Attack	What it does
Blackhole	A malicious node silently drops all traffic routed through it
Flooding	Overwhelms the network with excessive control messages
Rank	A node lies about its position in the routing tree to attract traffic
Version	Forces unnecessary, costly network-wide topology rebuilds

This report covers the full process: how the data was explored and cleaned, a data leakage bug that was found and fixed, a comparison across a baseline and two stronger models, and a diagnosis of exactly where the remaining errors come from and why.

2. Dataset

2.1 Source and Citation

IoT-RPL 2021: Cyber Attack Dataset Based on RPL Routing for IoT, by Dhifallah, Tarhouni, Moulahi, and Zidi. Mendeley Data, DOI [10.17632/4rcbbry2sc.1](https://doi.org/10.17632/4rcbbry2sc.1), CC BY 4.0. Published specifically to support IDS research on RPL-based IoT/6LoWPAN networks so researchers don't have to simulate the attacks themselves.

2.2 Structure and Class Distribution

One row per RPL control message (DIS / DIO / DAO), with columns describing protocol layers (`frame_proto` , `protocol` , `control_type`), the message type, and protocol-specific fields (`DOAGID` , `DIO_info` , `object_cont_pt` , etc.).

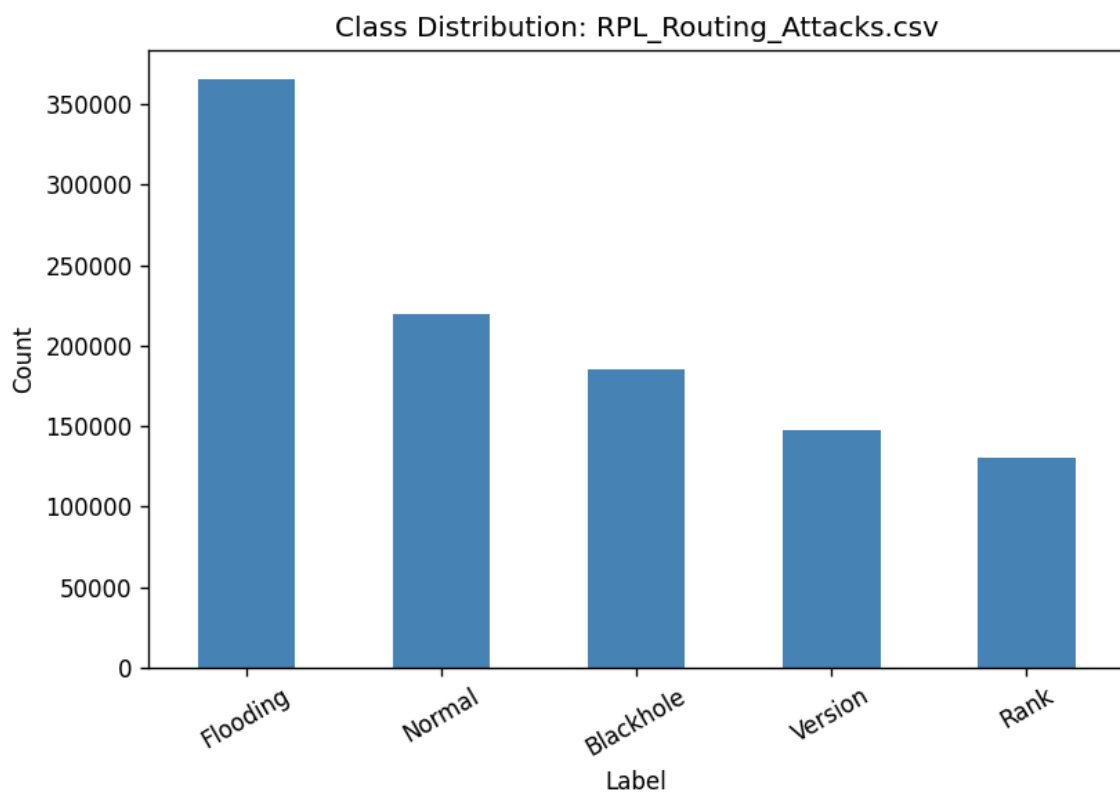


Figure 1. Class distribution: Flooding (365,603), Normal (219,470), Blackhole (185,749), Version (147,224), Rank (130,529). Notably imbalanced.

2.3 Scope of This Implementation

The full published dataset spans 10 files (`0.csv` through `9.csv` , 10,242,176 rows total). This project deliberately used one file (1,048,575 rows, about 10% of the full dataset) to keep iteration speed reasonable for a first version, not a limitation of the approach itself.

3. Implementation: Preprocessing Pipeline

3.1 Define

Before writing any code, the modeling target was defined: predict the `label` column (Normal / Blackhole / Flooding / Rank / Version), using every other column as an input feature. This is the standard framing for a supervised learning problem.

3.2 Load the Data

The dataset (1,048,575 rows, 23 columns) was loaded with `pandas.read_csv`. The file includes a real header row (`from` , `to` , `frame_proto` , ..., `label`), so no manual column naming was required.

3.3 Explore the Data

Initial exploration examined the raw rows directly rather than trusting assumptions:

- Confirmed the protocol structure: `IPv6` → `ICMPv6` → `RPL` → a specific message type (`DIS` , `DIO` , or `DAO`), with different message types populating different downstream fields.
- Checked label distribution and found significant class imbalance (Figure 1).
- Identified that several columns mix numeric values, hex codes (e.g. `AAAA0000`), and comma-separated lists (e.g. `to` , a neighbor list like `"4,10,11"`) depending on the row's message type, which ruled out a single blanket cleaning rule.

3.4 Clean the Data

For every feature column:

- List-type columns (comma-separated, e.g. `to`) were converted to a neighbor count instead of the raw list.
- Mostly-numeric columns (90%+ of values convert cleanly via `pd.to_numeric`) were kept as numbers, with remaining gaps filled with 0.
- Everything else (text/hex/protocol codes) was flagged as categorical, handled in feature selection below.
- Leftover `inf` / `-inf` values (from rate-based fields) were replaced with 0.

3.5 Feature Selection and Engineering

- Encoding: categorical feature columns were Label Encoded (text/hex to integer ID). The target label was also Label Encoded into a plain integer, with no further transformation needed since every model used here works directly with integer labels.
- Scaling: `StandardScaler` normalizes every feature to a comparable range, since protocol fields like `DOAG_info` reach into the billions while others are 0/1 flags.
- Train/test split: 80/20, stratified to preserve class proportions given the imbalance.

3.6 Detecting and Removing Data Leakage

The first training run scored a suspicious 100% accuracy across all 5 classes. Instead of reporting that as a win, the cause was investigated.

Finding: the `from` column (simulated device node ID) perfectly predicted the label. Every value mapped to exactly one class:

from value	Always labeled
1 to 11	Normal
12	Blackhole
13, 16	Flooding
14	Rank
15	Version

Each attacker device only ever ran one type of attack for the whole simulation, so the model was memorizing device identity instead of learning attack behavior. This would be useless in a real deployment where the malicious device is unknown in advance.

Fix: dropped `from` entirely as a feature. Accuracy fell to a realistic ~72 to 76%, not a regression, but the removal of a lie. This is treated as a feature-selection decision: the column was excluded specifically because it encodes simulation setup, not genuine attack behavior.

4. Model Implementation

4.1 Baseline Model: Logistic Regression

Before reaching for a more complex model, Logistic Regression (a basic linear statistical classifier) was trained first, to check how much of the signal in the data is simple versus how much needs a stronger model to capture.

4.2 Random Forest

100 trees, balanced class weights, trained on the same flat scaled features used for the baseline. A model built specifically for tabular data, added to see how much further it could push accuracy beyond Logistic Regression.

4.3 Gradient Boosting

`HistGradientBoostingClassifier` builds trees sequentially, with each one correcting the previous trees' errors. Added specifically to test whether a fundamentally stronger algorithm could break through the accuracy plateau Random Forest hit.

5. Results and Model Comparison

Model	Test Accuracy	Notes
Logistic Regression	62.82%	Basic statistical baseline
Random Forest (100 trees, balanced weights)	71.71%	Built for tabular data
Gradient Boosting	73.06%	Highest raw accuracy, but does not fix the real bottleneck, see Section 6

Random Forest and Gradient Boosting only beat Logistic Regression by about 8 to 10 points, meaning the extra model complexity buys a real but modest gain over a simple statistical model.

6. Error Analysis and Diagnosis

A confusion matrix was generated for the Random Forest model to identify exactly where its errors concentrate:

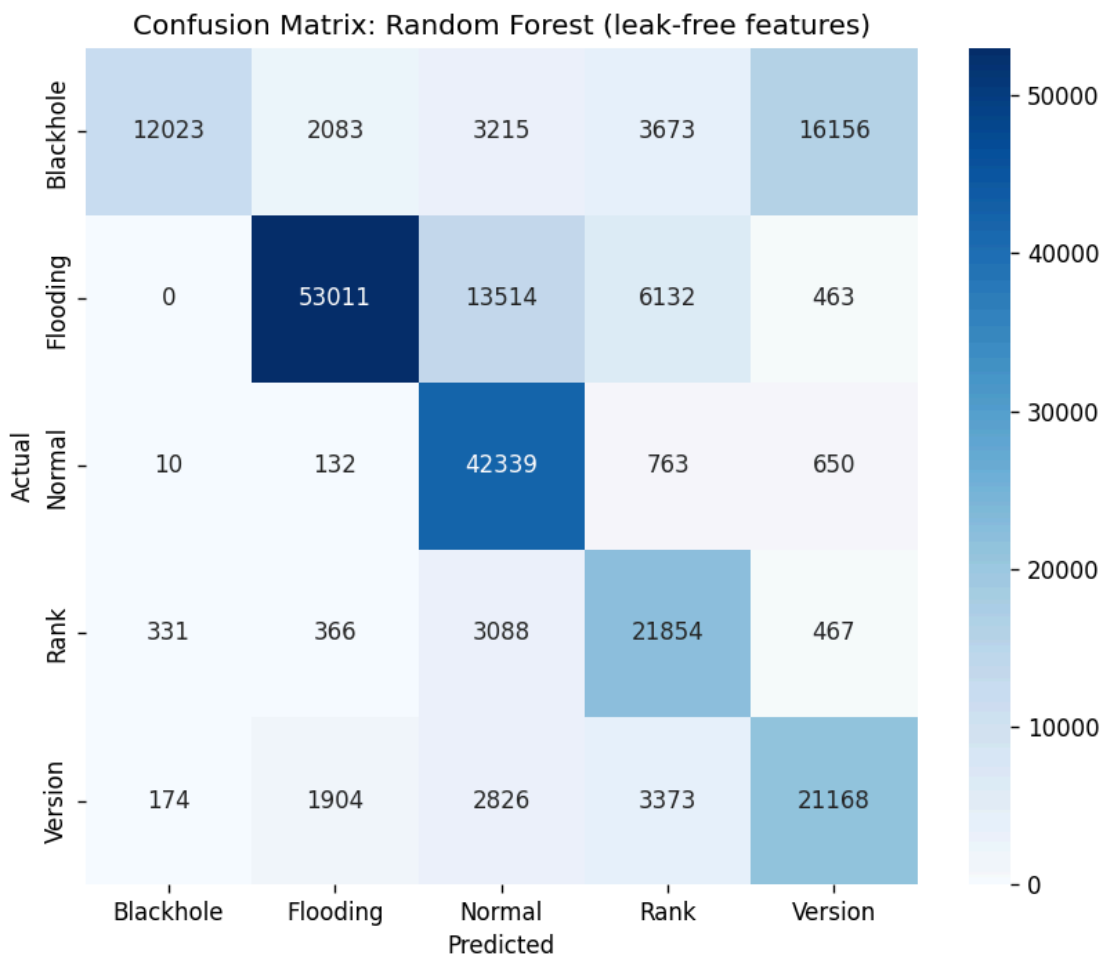


Figure 2. Confusion matrix (Random Forest). Rows = actual class, columns = predicted class.

Finding: of 37,150 real Blackhole attacks, 16,156 were misclassified as Version, the single largest error source. A column-by-column comparison of Blackhole vs. Version rows showed their real protocol-content fields are nearly identical between the two classes. The one column that does differ (`to` , neighbor list) encodes which simulated nodes were used per scenario, the same kind of leakage as `from` , not genuine attack behavior, so it was not used to fix this.

Conclusion: this is a data limitation, not a model limitation, confirmed by testing Random Forest and Gradient Boosting, two very differently built models, and finding the same confusion pattern in both. Gradient Boosting's higher accuracy did not fix the confusion, it shifted which class gets favored: Blackhole recall improved from 32% to 69%, while Version recall dropped from 72% to 29%.

A genuine fix would likely require sequence and time based features (e.g. how a node's claimed rank changes across its last several messages) rather than single-message snapshots. This is not cleanly supported by this

dataset, since there is no timestamp column, and the only per-node grouping key available is `from`, which would reintroduce the original leakage.

7. Key Takeaways

- **A perfect score is a red flag, not a win.** Always ask why before trusting a great result.
- **Switching models isn't a substitute for understanding the data.** Random Forest and Gradient Boosting, two very differently built models, hit the identical confusion pattern, which shows the bottleneck was the data, not the model.
- **A baseline matters.** Training Logistic Regression first confirmed the stronger models' accuracy was genuinely earned, not trivial.
- **Class imbalance matters.** It was handled through stratified splitting and class weighting, not ignored.
- **Not every improvement is real.** A raw accuracy increase can hide a trade-off (worse recall elsewhere) that only shows up in a per-class breakdown.
- **Simpler is often good enough.** Logistic Regression got within about 8 to 10 points of the stronger models, and a CNN tried early on never beat plain Random Forest despite far more code and setup, so it was dropped in favor of these simpler, better-suited models.